OMN

Team 04 Final Presentation

Robot Programing with ROS
Summer Semester 2026

Jun 03, 2026

Outline

Navigation: Main Steps

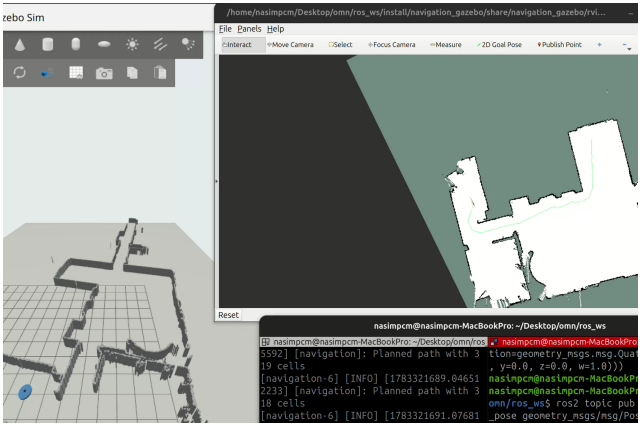
Planning

1. `timer_callback()` waits until a map, pose, and goal are available.
2. Robot and goal positions are converted from metres to map-grid cells.
3. A* plans a path using horizontal, vertical, and diagonal moves.
4. Obstacles are inflated to maintain clearance, and the grid path is smoothed to reduce zig-zag motion.

Path Following

5. Every $0.2\,\mathrm{s}$, a waypoint approximately $0.4\,\mathrm{m}$ ahead is selected.
6. A `TargetVector` pointing toward the waypoint is published.
7. When the goal is within $0.4\,\mathrm{m}$, the robot is stopped and `goal_succeeded` is reported.

A* Path Planning



Evaluation Function

$$f(n) = g(n) + h(n)$$

$g(n)$ is the path cost from the start and $h(n)$ estimates the remaining cost to the goal.

- Eight-connected grid search
- Straight-move cost: 1
- Diagonal-move cost: $\sqrt{2}$
- Corner-cutting is prevented
- Inflated cells add obstacle-clearance cost

A* Implementation

1. **Initialize:** add the start cell to the open-set priority queue, set $g(\text{start}) = 0$, and initialize `came_from`, `g_score`, and the closed set.
2. **Select:** remove the cell with the smallest estimated total cost $f(n)$.
3. **Check the goal:** if selected, reconstruct the path from the stored parent cells.
4. **Mark explored:** skip cells already processed; otherwise add the cell to the closed set.
5. **Generate neighbors:** consider all eight adjacent cells using straight and diagonal move costs.
6. **Apply obstacle costs:** reject blocked or invalid cells and penalize inflated cells near obstacles.
7. **Update better paths:** if a lower cost is found, update the neighbor's parent and scores, then add it to the open set.
8. **Finish:** return the reconstructed path, or report `no_path` if the open set becomes empty.

Next Waypoint Selection

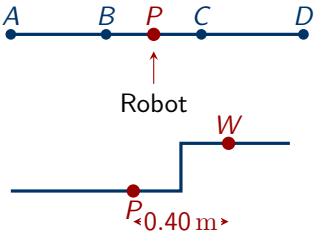
1. Locate the Robot on the Path

Project the robot onto every remaining path segment and select the closest point P.

2. Look Ahead Along the Path

Starting at P, measure $0.40\,\mathrm{m}$ along the path to obtain waypoint W. If the distance crosses a corner, continue on the next segment.

- `path_progress` prevents the selected position from jumping backward when the path passes close to itself.
- If less than $0.40\,\mathrm{m}$ remains, return the final goal point.



Look-ahead continues across corners

Requirements Identified

- Metrics to quantify localization and navigation performance (For anomaly detection)
- Anomaly detection (Single source and Multiple source)
- Feature vectors for Anomaly Prediction
- Anomaly Prediction FOL Rule Derivation
- Scenario based and log based predictions
- Implement as Python script (*.py) or package that includes the complete pipeline

Dataset Overview

Dataset Properties

- **Total Scenarios:** 100 directories
- **Runs per Scenario:** 3 (labeled 0, 1, 2)
- **Total Runs:** $\sim$300 experiments
- **Robot:** TurtleBot4 (0.22m diameter)

Environment Types

- door-width
- door-size
- room-size
- hallway-window

File Structure per Run

File	Content
poses.csv	Robot trajectories
behaviors.csv	BT execution logs
rosbag2.csv	ROS2 bag + AMCL
scenario.config	Goals, params
run.yaml	Run metadata

Pre-processing Steps

- **Data Synchronization**: Aligns the robot's estimated position with the "Ground Truth" (actual capabilities) data.
- **Stationary Filtering**: Removes irrelevant data where the robot was not moving, typically at the start or end of a run.
- **Coordinate & Orientation Normalization**:
- **Yaw Normalization**: Normalizes all yaw angles to the range $[-\pi, \pi]$.
- **Quaternion Conversion**: Converts orientation data from quaternion (w, x, y, z) to yaw angles for easier error calculation.

Performance Metrics (8 Metrics)

Metric	Unit	Description	Category
mean_pos_error	m	Average position error	Localization
rmse_pos	m	Root mean square error (spike sensitive)	Localization
mean_yaw_error	rad	Average orientation error	Localization
executed_path_length	m	Total path traveled	Path
path_efficiency	ratio	gt_path / executed (1 is best)	Path
mean_linear_velocity	m/s	Average movement speed	Kinematics
trajectory_smoothness	rad/s^2	Angular acceleration (lower = smoother)	Kinematics
duration	s	Total run time	Kinematics
mean_amcl_uncertainty	m	Average localization confidence	AMCL

Features for Causal Analysis (27 Features)

Anomaly Type	Primary Predictors	Secondary Predictors
Collision	near_wall, tight_clearance, near_static_obstacle, clearance_ratio	min_wall_distance, min_obstacle_distance
Stuck	in_narrow_corridor, in_small_room, tight_obstacle_clearance	corridor_width, room_area
Goal Failure	goal_near_wall, waypoint_in_tight_space, door_too_narrow	goal_wall_distance, door_width
Localization Uncertainty	high_noise, noise_level	—
Path Inefficiency	obstacle_clearance_ratio, num_obstacles, total_obstacle_area	min_door_narrow, goal_through_door

Anomaly Detection and Classification

Rule-Based Anomalies (8 Types)

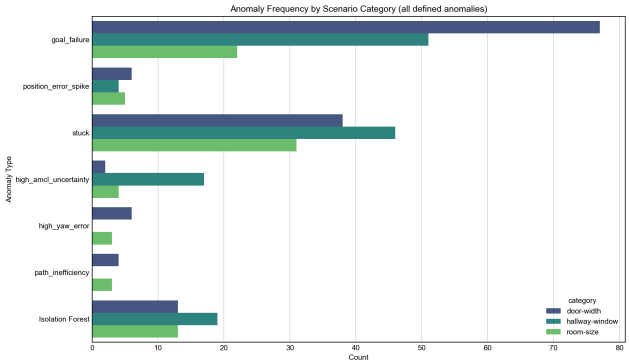
- **goal_failure**: Goal not reached
- **no_initiation**: Robot didn't start
- **position_error_spike**: Sudden errors
- **stuck**: Robot stopped moving
- **high_amcl_uncertainty**: Poor localization
- **high_yaw_error**: Orientation issues
- **path_inefficiency**: Sub-optimal path
- **oscillation**: Jerky motion (smoothness > 2.0)

ML-Based Anomaly Detection

Isolation Forest trained on 8 standardized metrics:

- Captures multi-metric abnormal patterns missed by rules
- Unsupervised learning for outlier detection

Anomaly Detection Results



Key Findings

- Total occurrences: **341**
- **goal_failure** is most common (50%)
- **stuck** follows at 38.3%
- **Isolation Forest** catches 15%
- Single run can have multiple anomalies

Anomaly Correlations Analysis

- **position_error_spike + high_yaw_error** (Jaccard = 0.60)
  → Poor orientation tracking leads to position drift

- **high_yaw_error + path_inefficiency** (Jaccard = 0.78)
  → Orientation errors cause inefficient path execution

Early Prediction: Predictability

Anomaly	PR-AUC	ROC-AUC	Cases
goal_failure	0.852	0.828	150
stuck	0.766	0.789	115
Isolation Forest	0.661	0.834	45

Early Multi-features and Anomaly Correlation

Stuck

- path_length: r=-0.491
- std_velocity: r=-0.437
- path_efficiency: r=-0.384

Goal Failure

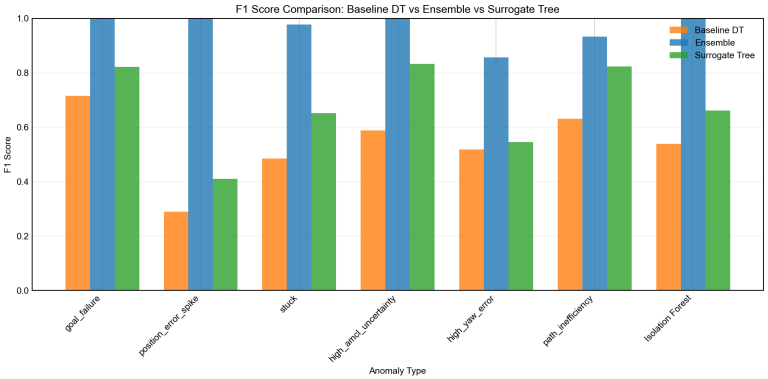
- duration: r=+0.524
- mean_angular_vel: r=-0.283
- path_length: r=+0.276

high_amcl_uncertainty

- rmse_pos: r=+0.291
- max_velocity: r=+0.277
- std_pos_error: r=+0.274

Methodology Evolution

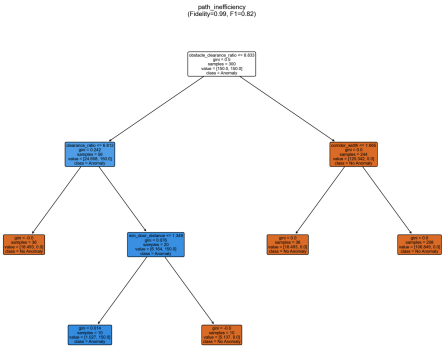
Simple Decision Tree → Ensemble Model → Surrogate Model



Current Approach

- **Step 1: High-Capacity Model Training (Ensemble)**
  - → Voting Classifier: Decision Tree + Random Forest + Gradient Boosting
  - → Separate trees for both log-based and scenario-based predictions
  - → Prioritizes F1-score
- **Step 2: Surrogate Model Training (Distilled Decision Tree)**
  - → Separate trees for both log-based and scenario-based predictions
  - → Constrained tree (max_depth=4)
  - → Target: Ensemble predictions (not ground truth)
- **Step 3: Rule Extraction**
  - → Extract paths with P(Anomaly) > 0.5
- **Step 4: FOL Translation**
  - → Map thresholds to semantic predicates

What is a Decision Tree / Surrogate Model?



Why Surrogate? Captures ensemble's knowledge in an interpretable tree structure, enabling FOL rule extraction while maintaining high fidelity ($>$90%).

Technical Insights (Important)

1. **Better quality rules with just scenario information**
   - In case of many anomalies it showed better predictive power and generalization (F1 score and other metrics) when just used scenario description, robot information, and environment JSON file for deriving FOL rules instead of also including more informative logs, CSV files.

2. **Surrogate Distillation**
   - Ensemble models achieve higher accuracy, but surrogate trees provide 90%+ fidelity with full interpretability.
   - Surrogate trees were even simpler than simple decision trees yet showed better results (F1, confusion metrics).

Domain and Methodological Insights

1. **Narrow Door Width is Critical**: Doors $< 1.8\times$ robot footprint cause $>60\%$ of goal failures
2. **Sensor Noise Cascades**: High noise (>0.05) correlates with both localization errors AND path inefficiency
3. **Corridor Navigation is Challenging**: Narrow corridors ($<3\times$ footprint) are disproportionately associated with stuck states
4. **Static Obstacles Compound Difficulty**: Scenarios with obstacles + narrow passages have $2\times$ failure rate
5. **Generalization Requires Feature Abstraction**: Relative features (clearance ratios) generalize better than absolute measurement numbers

Team 2 Final Presentation

Thank you!

Questions?